

Laborator 11

Comunicarea prin pipe-uri

11.1 Scopul lucrării

Laboratorul curent prezintă principiile comunicării între procese folosind pipe-uri. Se vor studia apelurile de sistem Linux pentru a lucra atât cu pipe-uri cu nume cât și cu pipe-uri anonime.

11.2 Fundamente teoretice

11.2.1 Principiile comunicării prin pipe-uri

Un *pipe* este un tip special de fișier, utilizat pentru comunicarea între procese. Operațiile uzuale pe fișiere (**read** și **write**) vor funcționa și în cazul pipe-urilor.

Pipe-urile funcționează după principiul FIFO (*First In, First Out*). Conceptual, putem să ne imaginăm un pipe similar cu o coadă, în care citirea se face la unul din capete, iar scrierea la celălalt. Principiul FIFO este strict, operația **lseek** pentru deplasarea cursorului în fișier nefiind permisă.

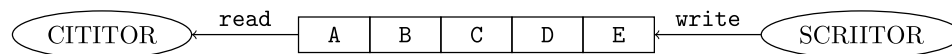


Figura 11.1: Ilustrarea principiului FIFO pentru un pipe

Figura 11.1 reprezintă un pipe în care s-au scris octeții A, B, C, D, E. La operația de scriere, se va scrie în dreapta lui E. Citirea se va face din partea stângă, începând cu A.

În condiții normale de utilizare, la fiecare din capetele unui pipe este conectat un proces (unul citește, celălalt scrie). Sistemul de operare permite aceluiași proces să facă ambele operații, dar acest caz se întâlnește mai rar în practică.

Dacă ambele capete sunt conectate, în momentul în care pipe-ul este gol, apelul de sistem **read** nu va returna 0, ca și în cazul unui fișier obișnuit, ci va bloca procesul apelant până ce datele cerute sunt scrise. La fel, dacă pipe-ul este plin (există o capacitate maximă) și sunt ambele capete conectate, apelul de sistem **write** va bloca procesul apelant până ce în pipe va fi suficient spațiu liber pentru scrierea datelor.

Dacă se încearcă citirea dintr-un pipe gol, dar nu avem nici un scriitor conectat, cititorul nu va mai fi blocat, ci operația **read** va returna 0.

Dacă se încearcă scrierea într-un pipe la care nu este conectat nici un cititor, procesul care apelează **write** va primi semnalul **SIGPIPE**, care în mod normal închide procesul respectiv (se poate preveni închiderea recepționând explicit semnalul **SIGPIPE**).

La deschiderea unui pipe cu nume folosind apelul de sistem **open**, fie în citire, fie în scriere, procesul curent este blocat până ce un alt proces se va atașa la celălalt capăt al pipe-ului, apelând **open** cu operația opusă. Cu alte cuvinte, operația **open** funcționează ca o barieră, ce nu permite proceselor cititor și scriitor să continue decât împreună.

11.2.2 Pipe-uri cu nume

```
#include <sys/types.h>
#include <sys/stat.h>

int mkfifo(const char *pathname, mode_t mode);
```

Apelul de sistem `mkfifo` este utilizat pentru a crea un pipe cu nume, primind ca prim argument calea spre acesta, respectiv permisiunile de acces, ca al doilea argument. În caz de succes se returnează 0.

11.2.2.1 Exemplu de utilizare a pipe-urilor cu nume

Vom scrie două programe, `writer` și `reader`. Primul va crea un pipe cu nume, îl va deschide și va scrie un întreg, apoi îl va închide și îl va șterge. Procesul `reader` va deschide pipe-ul și va citi întregul scris de `writer`.

<u>writer.c:</u>	<u>reader.c:</u>
1 #include <stdio.h>	1 #include <stdio.h>
2 #include <sys/types.h>	2 #include <sys/types.h>
3 #include <sys/stat.h>	3 #include <sys/stat.h>
4 #include <fcntl.h>	4 #include <fcntl.h>
5 #include <unistd.h>	5 #include <unistd.h>
6	6
7 #define FIFO_NAME "l11_my_fifo"	7 #define FIFO_NAME "l11_my_fifo"
8	8
9 int main(void)	9 int main(void)
10 {	10 {
11 int fd = -1;	11 int fd = -1;
12 int x = 42;	12 int x = 0;
13	13
14 //create fifo	14 //open, read and close fifo
15 if(mkfifo(FIFO_NAME, 0600) != 0){	15 fd = open(FIFO_NAME, O_RDONLY);
16 perror("Err creating FIFO");	16 if(fd == -1) {
17 return 1;	17 perror("Could not open FIFO");
18 }	18 return 1;
19	19 }
20 //open, write and close fifo	20 read(fd, &x, sizeof(x));
21 fd = open(FIFO_NAME, O_WRONLY);	21 printf("The read value is: %d\n", x);
22 if(fd == -1) {	22 close(fd);
23 perror("Could not open FIFO");	23
24 return 1;	24 return 0;
25 }	25 }
26 printf("Writing value %d\n", x);	
27 write(fd, &x, sizeof(x));	
28 close(fd);	
29	
30 //delete fifo	
31 unlink(FIFO_NAME);	
32	
33 return 0;	
34 }	

Programul `writer` începe prin a crea pipe-ul, apelând `mkfifo` la linia 15. În caz de reușită se continuă deschizând pipe-ul pentru scriere (linia 21), ca și cum ar fi un fișier obișnuit. După deschidere, se scrie în pipe folosind apelul de sistem `write` (linia 27), tot similar cu operația pe un fișier. Pipe-ul este închis la linia 28, apoi șters la linia 31 folosind apelul de sistem `unlink`.

Nu trebuie să ne facem griji dacă pipe-ul mai este utilizat încă de către *reader*, pentru că dacă la ștergere un fișier este deschis de anumite procese, deși numele lui dispăre din arborele de fișiere, inode-ul și datele lui sunt păstrate până când fișierul este închis de către toate procesele. Singura problemă ar fi dacă pipe-ul ar fi șters de *writer* înainte de a fi deschis de *reader*, lucru care, însă, nu se poate întâmpla din motive explicate puțin mai jos.

Programul *reader* nu mai creează sau șterge pipe-ul, deoarece aceste operații sunt efectuate de *writer*. Pipe-ul este deschis în citire la linia 15, se citește întregul la linia 20, apoi se închide la linia 22.

După ce compilăm cele două programe, ar trebui executat întâi *writer*. Acesta creează pipe-ul, apoi se blochează la apelul `open` de la linia 21. Sistemul de operare nu ne permite să trecem mai departe, până nu se deschide și celălalt capăt al pipe-ului, în citire. Continuăm executând și programul *reader* într-un alt terminal. Atunci când se apelează `open` (linia 15), pipe-ul va avea ambele capete deschise și ambele programe pot continua. Din acest moment programele continuă într-o ordine arbitrară, fie *writer* ajunge primul la apelul `write`, fie *reader* ajunge primul la `read`. În al doilea caz, programul *reader* va fi blocat până ce datele sunt scrise în pipe de către *writer*.

11.2.3 Pipe-uri anonime

```
#include <unistd.h>

int pipe(int pipefd[2]);
```

Pipe-urile anonime nu au un nume după care să fie identificate, acestea fiind accesibile doar prin descriptorii de fișier obținuți prin apelul de sistem `pipe`. Acesta primește ca argument adresa unui vector cu două elemente de tip `int`, în care va completa doi descriptori de fișier: `pipefd[0]` va fi folosit pentru citire, în timp ce `pipefd[1]` va fi folosit pentru scriere.

Pipe-urile anonime nu se pot utiliza decât pentru comunicarea dintre un părinte și un copil. Părintele va crea pipe-ul și va obține cei doi descriptori de fișier (pentru citire și scriere). Ulterior, va efectua apelul de sistem `fork` pentru a da naștere procesului copil. Copilul va moșteni structurile de date ale părintelui, inclusiv descriptorii de fișiere și astfel va putea comunica prin pipe-ul creat.

11.2.3.1 Exemplu de utilizare a pipe-urilor anonime

Vom relua exemplul din secțiunea 11.2.2.1, în care două procese comunică printr-un pipe, de data asta anonim, primul proces transmițând o valoare întreagă celui de-al doilea.

Pentru a comunica printr-un pipe anonim, cele două procese vor trebui să fie în relația părinte-copil.

```
1 int main(void)
2 {
3     int fd[2];
4     int x = 0;
5
6     if(pipe(fd) != 0) {
7         perror("Could not create pipe");
8         return 1;
9     }
10
11     if(fork() != 0) {
12         //parent
13         close(fd[0]); //no use for read end
14         x = 42;
15         write(fd[1], &x, sizeof(x));
16         printf("Parent: wrote %d to pipe\n", x);
17         close(fd[1]);
18         wait(NULL);
```

```

19     } else {
20         //child
21         close(fd[1]); //no use for write end
22         read(fd[0], &x, sizeof(x));
23         printf("Child: read %d from pipe\n", x);
24         close(fd[0]);
25     }
26
27     return 0;
28 }

```

Programul începe prin construcția pipe-ului. Va trebui să declarăm un vector de doi întregi, care vor stoca descriptorii de fișier pentru cele două capete ale pipe-ului (linia 3). Procesul părinte inițializează pipe-ul prin apelul de sistem `pipe` la linia 6. În acest moment, descriptorii de fișier sunt scriși în vectorul `fd`.

La linia 11, efectuăm apelul de sistem `fork`, ce dă naștere procesului copil. Dacă `fork` returnează o valoare nenulă suntem în procesul părinte, altfel în copil.

Ambele procese continuă prin a închide capătul nefolosit al pipe-ului. Procesul părinte va scrie în pipe, deci nu va avea nevoie de capătul de citire și va închide `fd[0]` (linia 13). Analog, procesul copil doar citește, deci poate închide capătul de scriere `fd[1]` (linia 21).

Pentru a scrie valoarea întreagă în pipe, procesul părinte va folosi apelul de sistem `write` (linia 15), apoi va închide capătul de scriere al pipe-ului (linia 17) și va aștepta terminarea procesului copil (linia 18).

Similar, procesul copil va citi întregul din pipe (linia 22), apoi va închide capătul de citire (linia 24).

11.2.4 Comunicarea bidirecțională

În exemplele anterioare, am observat o comunicare unidirecțională între două procese. În exemplul din secțiunea 11.2.2.1, procesul `writer` transmitea un întreg spre procesul `reader`, în timp ce în exemplul din secțiunea 11.2.3.1 părintele transmitea întregul spre copil. În multe situații din lumea reală este necesar să se comunice în ambele sensuri, un proces putând atât să scrie cât și să citească mesaje.

Deși este posibil să se comunice bidirecțional folosind un singur pipe, sincronizând explicit operațiile de scriere și citire (folosind semafoare), este mult mai simplu să comunicăm bidirecțional folosind două pipe-uri. Dacă folosim un singur pipe pentru comunicare bidirecțională fără să sincronizăm comunicarea explicit, se poate întâmpla ca un proces să citească datele pe care tot el le-a scris, lucru pe care în general nu îl dorim.

În exemplul de mai jos, două procese părinte și copil comunică din nou folosind pipe-uri anonime. De data asta, comunicarea va fi bidirecțională. Părintele inițializează o variabilă de tip caracter pe care o transmite copilului. Urmează 10 iterații în care fiecare proces primește litera, o afișează, apoi transmite litera următoare interlocutorului.

```

1  int main(void)
2  {
3      int fdP2C[2], fdC2P[2];
4      char c = 0;
5      int i;
6
7      if(pipe(fdP2C) != 0 || pipe(fdC2P) != 0) {
8          perror("Could not create pipes");
9          return 1;
10     }
11
12     if(fork() != 0) {
13         //parent
14         close(fdP2C[0]);

```

```

15     close(fdC2P[1]);
16     c = 'a';
17     for(i=0; i<10; i++) {
18         write(fdP2C[1], &c, sizeof(c));
19         read(fdC2P[0], &c, sizeof(c));
20         printf("Parent: %c\n", c);
21         c++;
22     }
23     close(fdP2C[1]);
24     close(fdC2P[0]);
25     wait(NULL);
26 } else {
27     //child
28     close(fdP2C[1]);
29     close(fdC2P[0]);
30     for(i=0; i<10; i++) {
31         read(fdP2C[0], &c, sizeof(c));
32         printf("Child: %c\n", c);
33         c++;
34         write(fdC2P[1], &c, sizeof(c));
35     }
36     close(fdP2C[0]);
37     close(fdC2P[1]);
38 }
39
40 return 0;
41 }

```

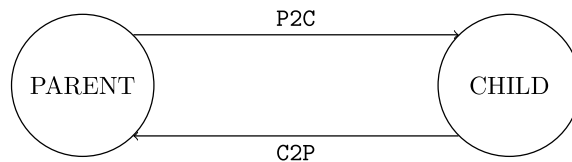


Figura 11.2: Comunicarea bidirecțională între două procese

Figura 11.2 ilustrează modul de comunicare între procesele părinte și copil din exemplul curent. Se folosesc două pipe-uri, pe care le vom numi P2C (*parent to child*) și C2P (*child to parent*).

La linia 7 se inițializează cele două pipe-uri anonime, iar la linia 12 se apelează `fork` pentru crearea procesului fiu. Ambele procese încep prin a închide capetele nefolosite ale pipe-urilor (liniile 14–15 și 28–29).

Procesul părinte inițializează variabila `c` cu litera `'a'` (linia 16), apoi începe o buclă de 10 iterații în care:

- transmite litera către copil prin pipe-ul P2C (linia 18);
- citește litera primită de la copil prin pipe-ul C2P (linia 19);
- afișează litera (linia 20);
- incrementează litera (linia 21).

Procesul copil conține o buclă similară, dar care începe cu citirea literei de pe pipe-ul P2C (linia 31) și se termină cu trimiterea literei incrementate către părinte, prin pipe-ul C2P (linia 34).

Executând programul se poate observa că cele două procese ajung să afișeze alternativ primele 20 de litere din alfabet (de la `'a'` la `'t'`).

11.3 Mersul lucrării

11.3.1 Întrebări recapitulative

1. În ce ordine sunt citați octeții scriși într-un pipe?
2. Ce se întâmplă când încercăm să citim dintr-un pipe gol, care are un scriitor conectat? Dar dacă nu există un scriitor conectat?
3. În ce situații preferăm să folosim un pipe cu nume față de unul anonim?
4. Ce primește ca argument apelul de sistem `pipe` și la ce este utilizat acest argument?
5. Cum pot două procese să comunice bidirecțional?

11.3.2 Probleme rezolvate

1. (*l1p1_writer.c* și *l1p1_reader.c*) Să se scrie două programe care comunică printr-un pipe cu nume, primul transmitând o valoare întreagă către al doilea. Programele sunt descrise în secțiunea 11.2.2.1.
2. (*l1p2_unnamed.c*) Să se scrie un program care dă naștere unui proces copil cu care comunică printr-un pipe anonim, transmitând o valoare întreagă. Programul este descris în secțiunea 11.2.3.1.
3. (*l1p3_bidir.c*) Să se scrie un program care creează un proces copil cu care comunică bidirecțional prin pipe-uri anonime, transmitând o literă de la unul la altul și incrementând-o. Programul este descris în secțiunea 11.2.4.

11.3.3 Probleme propuse

1. (25p) Modificați exemplul din secțiunea 11.2.3.1 astfel încât să se transmită un string de dimensiune variabilă în loc de întreg. Ca soluții posibile ar fi să citați de pe pipe caracter cu caracter până la terminatorul `'\0'` sau să transmiteți înainte de șirul propriu-zis lungimea lui, ca un întreg (reprezentat pe număr fix de octeți).
2. (25p) Folosind informația că un proces este blocat atunci când scrie într-un pipe plin, scrieți un program care determină dimensiunea maximă pentru un pipe cu nume și pentru un pipe anonim.
3. (25p) Scrieți două programe *client.c* și *server.c*. Într-o buclă, clientul va citi doi întregi și o operație de efectuat (`'+'` sau `'-'`) și le va transmite printr-un pipe server-ului. Server-ul va efectua operația și va transmite rezultatul către client, care îl va afișa pe ecran. Clientul se poate termina când operatorul introdus este `'x'`.
4. (25p) Implementați comunicarea bidirecțională din exemplul din secțiunea 11.2.4 utilizând un singur pipe în loc de două. Evitați situația ca un proces să citească datele scrise de el însuși în pipe, sincronizând explicit comunicarea.